

3교시: 표준 실습 앱 소스코드 확인

수업 목표

- 표준 실습 앱의 file tree와 build context를 확인한다.
- .dockerignore 가 build context와 보안/효율에 미치는 영향을 설명한다.
- Dockerfile build를 실행하기 전 path와 파일 존재를 검증한다.

50분 흐름

시간	활동	비중	학생 산출
0-8분	실습 앱 위치 확인	실행 15%	path evidence
8-20분	source tree 읽기	실행 25%	file tree note
20-32분	.dockerignore 확인	실행 20%	ignore note
32-42분	local file 내용 확인	실행 20%	source evidence
42-50분	build context 위험 정리	설명 20%	risk note

Visual 1: 표준 실습 앱 소스 구조

Week 2
Day 2

표준 실습 앱 소스 트리 & 로컬 구조 확인

Lesson 3

소스 구조 (프로젝트 루트)

practice-app/

public/

index.html

styles.css

Dockerfile

.dockerignore

README.md

logs/

node_modules/

screenshots/

역할 요약

- index.html: 메인 페이지
- styles.css: 스타일
- Dockerfile: 이미지 빌드 정의
- .dockerignore: 불필요 파일 제외
- README.md: 실행 방법 기록

로컬 확인 (기본 동작 확인)

브라우저 미리보기

← → ↻ http://localhost:8080



Docker 실습 앱

Docker로 배포하는 첫 번째 웹 페이지입니다.

시작하기

로컬 실행 예 (정적 파일 서버)

\$ npx http-server public -p 8080

브라우저에서 http://localhost:8080 접속 확인

확인 체크리스트

- ✓ 페이지가 정상 표시된다.
- ✓ 스타일이 적용되어 있다.
- ✓ 콘솔 오류가 없다.

빌드 컨텍스트

Docker 빌드 시 전송되는 파일 범위

포함 (전송됨)

public/

index.html

styles.css

Dockerfile

.dockerignore

README.md

제외 (.dockerignore)

logs/

node_modules/

screenshots/

*.log

.DS_Store

.git/

*.tmp

이 파일들만 이미지 빌드

README (실행 기록 & 가이드)

- 목적: 실행 방법과 명령을 기록해 재현 가능하게 함
- 예시 명령: 로컬 실행, Docker 빌드/실행
- 문제 해결 팁과 확인 방법 기록

이 이미지는 index.html, styles.css, Dockerfile, .dockerignore, README.md 가 build context 안에서 어떤 역할을 하는 지 보여준다.

실습 명령

```
cd week2/day2/labs/static-site
pwd
ls -la
```

```
sed -n '1,120p' Dockerfile
sed -n '1,120p' .dockerignore
```

기대 파일

파일	역할
index.html	nginx가 제공할 HTML
styles.css	정적 사이트 스타일
Dockerfile	image build instruction
.dockerignore	build context 제외 규칙
README.md	build/run/check/cleanup 예시

.dockerignore

```
.git
.DS_Store
node_modules
*.log
screenshots
README.local.md
```

.dockerignore 는 build context에 불필요한 파일이 들어가지 않게 한다. 이것은 build 속도뿐 아니라 secret/log/screenshot 노출 방지와 연결된다.

build context를 학술적으로 보기

build context는 Dockerfile이 보는 "입력 데이터 집합"이다. 일반적인 프로그램이 source file과 dependency를 입력으로 받아 binary를 만들듯, Docker build는 Dockerfile과 context를 입력으로 받아 image를 만든다. 따라서 context boundary가 불명확하면 build 결과도 불명확해진다.

`docker build .` 에서 마지막 `.` 은 현재 directory를 build context로 보낸다는 뜻이다. Dockerfile의 `COPY index.html styles.css ./` 는 이 context 안의 파일만 참조한다. 상위 directory의 secret file이나 운영 자료를 마음대로 복사할 수 없고, 복사하려고 하면 path error가 발생한다. 이 제약은 불편함이 아니라 재현성과 안전을 위한 boundary다.

context risk table

위험	예시	완화
secret 포함	.env, token file, private key	.dockerignore, secret scanning
build 느려짐	node_modules, screenshots, large logs	context 최소화
경로 재현 실패	다른 학생은 파일 위치가 다름	repo 기준 상대 경로 사용
cache 혼동	불필요한 파일 변경으로 cache invalidation	COPY 범위를 좁힘

evidence로 볼 output

4교시 build output에서 다음 줄을 반드시 읽는다.

```
#4 [internal] load build context
```

```
#4 transferring context: 2.42kB
```

context가 몇 MB, 몇 GB로 커지면 실습 앱 기준으로는 이상 신호다. 이때 `.dockerignore` 와 build directory를 먼저 확인한다.

핵심 유의사항

3교시는 겉으로는 파일 확인 시간처럼 보이지만 실제로는 build boundary를 이해하는 시간이다. `docker build .` 의 마지막 점을 그냥 관습으로 외우면 이후 `COPY` 오류를 계속 낸다. 마지막 `.` 은 현재 directory 전체를 build context로 보내겠다는 의미다.

실습을 시작하기 전에 반드시 `pwd` 를 기록한다. 같은 명령이라도 어디서 실행했는지에 따라 build context가 달라진다. Docker 초급 오류의 상당수는 Dockerfile 문법 문제가 아니라 현재 directory가 틀린 문제다.

`.dockerignore` 는 "용량 줄이는 파일" 정도로 이해하면 부족하다. secret, log, screenshot, local config가 image build 과정으로 들어가지 않게 막는 보안 장치다. 특히 repository에 개인 token이나 cloud credential이 들어간 적이 있다면 이 지점을 강하게 인식해야 한다.

자주 놓치는 파일 경계

상황	학생이 놓치는 것	확인 명령
root에서 build해야 하는데 하위 폴더에서 실행	context가 너무 좁아짐	<code>pwd, ls -la</code>
하위 폴더에서 build해야 하는데 root에서 실행	context가 너무 넓어짐	<code>build output context size</code>
Dockerfile이 없는 위치에서 build	build definition을 찾지 못함	<code>ls Dockerfile</code>
.dockerignore가 누락됨	log/cache/secret이 context에 들어갈 수 있음	<code>sed -n '1,120p' .dockerignore</code>
<code>COPY ../file</code> 을 시도	context 밖 파일 접근	Dockerfile path와 context boundary

build 전 확인 루틴

build 전에는 아래 5줄 루틴을 고정한다.

```
pwd
ls -la
sed -n '1,120p' Dockerfile
sed -n '1,120p' .dockerignore
ls -lh index.html styles.css
```

이 루틴의 목적은 "명령을 많이 치는 것"이 아니라 build 입력을 명시하는 것이다. build 실패 후에야 파일 위치를 찾는 습관보다, build 전에 입력을 확인하는 습관이 운영적으로 더 좋다.

context 크기 해석

Day 2 사전 테스트에서 build context는 2.42kB 로 나왔다. 이 숫자는 실습 앱이 작기 때문에 정상이다. 만약 학생 output에서 수 MB 이상이 보이면 다음을 의심한다.

context 증가 원인	왜 문제인가	조치
node_modules 포함	파일 수가 많아 build 전송과 cache 판단이 느려짐	<code>.dockerignore</code> 추가
screenshot/video 포함	image와 무관한 큰 파일이 daemon으로 전달됨	별도 asset 경로 또는 ignore
.git 포함	history와 내부 정보가 전달될 수 있음	<code>.git ignore</code> 확인

log file 포함	민감 정보와 불필요한 변경으로 cache invalidation	*.log ignore
local README/config 포함	학생마다 다른 파일이 build 입력이 됨	필요한 파일만 COPY

학생 기록 템플릿

Lesson 3 Build Context Evidence

- 현재 directory:
- Dockerfile 존재 여부:
- build 대상 source file:
- .dockerignore 규칙:
- build context 예상 크기:
- context에 들어가면 안 되는 파일:
- 실제 build output의 context size:

다음 교시 연결 질문

3교시가 끝날 때 다음 질문을 남긴다.

지금 확인한 파일 중 image 안에 들어가는 파일은 무엇이고, Dockerfile을 만들기 위해서만 필요한 파일은 무엇인가?

정답은 index.html, styles.css 는 COPY 로 image에 들어가고, Dockerfile, .dockerignore, README.md 는 build를 설명하거나 제어하지만 현재 Dockerfile 기준으로 image 내부에 복사되지는 않는다는 것이다. 이 부분이 4교시 build output의 COPY step을 읽는 기준이 된다.

마무리 점검

마지막에는 자기 terminal 기준으로 build context를 명확히 가리킬 수 있어야 한다. "이 폴더가 daemon으로 전달된다"는 감각이 생겨야 COPY 오류와 secret 유입 위험을 줄일 수 있다.

제출 문장:

오늘 build context는 ____이고, image에 실제 복사되는 파일은 ____이다.
context에 들어가면 안 되는 파일 예시는 ____이다.

평가 기준

기준	2점 evidence
path 확인	실습 앱 directory에서 pwd를 기록했다.
file tree	5개 핵심 파일의 역할을 설명했다.
ignore 이해	.dockerignore가 제외하는 대상을 설명했다.
build 준비	Dockerfile과 source file 존재를 확인했다.

공식 근거 링크

- Docker build context: <https://docs.docker.com/build/concepts/context/>
- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>
- OWASP Secrets Management Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html